Solo, Like a Team (preview)

Claude Code Multi-Agent Orchestration in Practice

Make Claude the commander of a codex · GLM · Gemini worker pool — spend quota on judgment, run implementation in parallel

v1.0 · 2026

About this book

This book describes a working operating system for running one Claude Code session as the commander of a parallel worker pool — codex, GLM, and Gemini CLI. Every rule here came out of real operation in July-August 2026, including the incidents and how they were fixed.

> This book does not promise results or revenue from any tool. Model IDs and CLI flags are current as of 2026-08; when the tools change, an updated edition is free from the same purchase link. Account creation, payments, and secret handling must always be done by a human.

Buyer files: this PDF and `templates.zip` (orchestration section for CLAUDE.md, `worker_launch.sh`, `worker_watch.sh`, `glm_wrapper.sh`, review-gate prompt, task-spec and progress-report templates, safety denylist). 14-day, no-questions refund.

Contents

0. Introduction: Why a “Commander + Worker Pool”?

> **Revision — 2026-09-09:** Completion now uses harness exit code plus reading the result body, with notification-based waiting; marker gates and polling were retired to prevent false STALL decisions.

You open Claude Code and tell it, “Build this feature.” At first, it writes code as if by magic. But what happens when the project grows to ten files, then twenty, and the debugging loop gets longer? Before long, you see a message saying, “You’ve reached your weekly usage limit.” Or the agent sits idle for twenty minutes, thinking on its own. The context window becomes polluted with earlier failed attempts, and the model starts changing correct code into incorrect code.

Having one session handle everything works for toy projects, but it quickly reaches its limits in production. Relying on a single model creates three fundamental problems. First is the limit problem. If you waste a high-performance model on scanning code and routine typing, its allowance will be gone when you need deep reasoning. Second is latency. While the agent reads and edits files, the person can only watch the screen. Third is context pollution. As failed attempts and unrelated file contents accumulate, the model’s judgment deteriorates rapidly.

This book is an **operations manual you can copy, paste, and run tonight** to overcome those limits. The core idea is simple. Promote Claude from a coding laborer to a **commander (orchestrator)** that plans, delegates, and verifies. Delegate the actual coding and exploration to multiple worker models such as codex, GLM, and agy, running in parallel.

Three Benefits of Separating the Commander from the Workers

1. Distributed limits and parallel throughput. Reserve Claude’s weekly allowance for high-value judgment: planning, review, and coordination. Let workers such as codex and GLM handle the heavy work of reading and writing thousands of lines of code, each within its own execution limits and context. When independent tasks run at the same time and integration costs are controlled, implementation can proceed in parallel. This book covers both the required conditions and how to measure them.

2. A clean context. The commander Claude’s session contains only what needs to be done and the results summarized by the workers. It does not accumulate the waste text produced during intermediate debugging. Each worker has an independent process and context, so one worker’s hallucination does not affect the entire system. If a worker fails, discard it and start another one.

3. A foundation for controlled parallel scaling. This structure is parallel by design. You can modify an API endpoint, update the frontend UI, and write a database migration script at the same time. But if

you also overlap work that touches the same files and state, the cost of resolving conflicts consumes the benefit. The unit of parallelization is not the number of workers. It is the **number of tasks that can be verified independently**.

Comparison	Single session	Commander + worker pool
Judgment and implementation	Mixed in one context	Commander and execution contexts are separate
Waiting	The next task starts only after the current one finishes	Independent tasks run at the same time
Failure scope	The entire session can become polluted	Only the failed task is retried
Evidence of completion	Easy to rely on the model's completion statement	Check the process, logs, and tests together
Integration cost	Low, but throughput is constrained by one session	Can be high, so specifications and verification gates are required

The Minimum Operating Loop to Use from Day One

The commander writes completion conditions, not the work itself

[Why] If you pass a broad request such as "Improve the login feature" directly to a worker, it may ask questions, change unrelated files, or stop without running tests. In parallel execution, ambiguity is replicated across every worker.

[Rule] For the first run, fix only the goal, files to read, files to modify, work not to do, verification command, and completion-report format. Break each independent task into a size whose result can be judged within 3-15 minutes. If you still cannot judge it, do not add workers. Make the specification smaller.

[Command/Code] Submit the following command through the Claude Code Bash harness with `run_in_background: true`. Prepare directories first; the Perl-wrapped worker must be the final command. The following is the smallest call for checking a single documentation file. The `-m` and `-c` options were verified against the `codex exec --help` output stored with this book.

```
cd ~/project/myapp
mkdir -p docs/logs
perl -e 'alarm shift; exec @ARGV' 1800 codex exec -m gpt-5.6-luna -c
    model_reasoning_effort="low" \
  "TASK: Review the README installation commands. SCOPE: Modify README.md only.
    OUT-OF-SCOPE: Do not modify code. VERIFY: Review the README commands for shell syntax.
    REPORT: Last line DONE: README review" \
  < /dev/null > docs/logs/readme-worker.log 2>&1
```

The body must contain the **conclusion itself**. The following codex example includes optional `tokens used` diagnostic text; it is never a completion requirement.

```
Verified: npm install, npm test
Changed: 1 outdated script name in README.md
DONE: README review
tokens used
```

[Failure signal] The process has exited, but the log stops at "I will proceed once you approve the plan," without an implementation result. The opposite is also a misjudgment: declaring failure because the process is still alive but the log is small.

[Recovery] Read the harness notification exit code and the result body first. If the worker is waiting for approval, do not repeat the same command unchanged. Add "Do not ask questions; implement immediately. Do not wait for approval" to the beginning of the specification, fill in the missing completion conditions, and make one changed retry. If it fails for the same reason a second time, transfer the task to another worker.

Following this small loop alone greatly reduces the first common incident: confusing an exited process with a completed task.

```
# After the harness notification, record its exit code and read the result.
cat docs/logs/readme-worker.log
```

How This Book Is Structured and How to Read It

This is not a conceptual guide. It is a collection of CLAUDE.md rules and Bash scripts refined through painful experience running production systems. Every chapter is organized around practical operation.

Every core rule follows this sequence:

1. **[Why]**: Why the rule is needed, usually based on a painful failure
2. **[Rule]**: The clear principle to follow
3. **[Command/Code]**: CLI commands and scripts you can actually copy and paste
4. **[Failure signal]**: The symptoms that appear when the rule is violated
5. **[Recovery]**: How to restore the system after failure

You do not need to read the book from cover to cover. Complete the setup in Chapter 2, copy the templates from Chapter 12 into your project root, and start Claude Code as the commander. When you encounter a problem, return to the relevant chapter.

Note 1: Model names in the text, including `gpt-5.6-sol`, `glm-5.3`, and `claude-fable-5`, are current as of August 2026. Always replace them with the latest model names available when you read this book.

Preview: Three Real Incidents Written in Blood

Every rule in this book came from a real incident, not from theory. Throughout the book, we examine the full details and remedies for the following three major incidents.

- **2026-08-18 stall incident**: A worker stopped without a completion report, but the commander mistook it for completion and lost several hours. This became the lesson for how to monitor workers in Chapter 5.
- **2026-08-19 misclassification incident**: A non-codex worker's normal exit was misclassified as an error, stopping all work. It demonstrated the importance of worker-specific behavior and reading the result body in Chapters 5 and 7.
- **2026-08-20 Chrome profile-loss incident**: Browser automation touched the user's default profile, and approximately 2,000 bookmarks disappeared. This critical incident became the absolute standard for isolated, safe environments in Chapter 9.

We built on these incidents. The goal is not to be perfect on day one. It is to detect failures quickly and encode rules and checks so the same failure cannot recur. We will now move to Chapter 1 and begin by separating the roles of commander and worker clearly.

Checklist

- [ ] Are you currently using one model for all coding and running short of usage allowance?
- [ ] Do you wait while watching the screen as the agent writes code?
- [ ] After a long session, does the model forget earlier instructions or modify unrelated code?
- [ ] Do you often need to change several files or modules at the same time?
- [ ] Are you comfortable with background processes and terminal environments such as zsh and bash?
- [ ] Are you ready to distinguish a worker's completion statement from actual verification success?

Three common failures

1. **Still asking Claude to write the code**: Do not put a hammer in the commander's hand. Provide the specification and call a worker.
2. **Running workers only one at a time**: This discards the benefit of parallelism. Always run independent tasks at the same time.
3. **Ignoring templates and rules and relying on verbal instructions**: Without explicit task contracts, harness notifications, and result review, workers soon become uncontrollable.

1. Role Separation: Claude as the Commander, Everyone Else as the Hands

> **Revision — 2026-09-09:** Completion now uses harness exit code plus reading the result body, with notification-based waiting; marker gates and polling were retired to prevent false STALL decisions.

When developing alone, it may initially seem convenient to give one agent responsibility for planning, file exploration, implementation, testing, review, and commits. On long tasks, however, that convenience returns as waiting time, context pollution, approval waits, and missed verification. The purpose of this chapter is not to rank models. It is to build an operating structure in which one session retains judgment while multiple workers act as its hands and feet.

Claude is the commander in this structure. The commander decides what to build, divides the work into small tasks, and confirms whether the result is actually correct. codex, GLM, and agy are workers. Within a defined scope, workers read, edit, test, and return results. This brings the distinction between a human team lead and implementers into one person's terminal.

This separation is not about authority. It is about detecting failures quickly, spending expensive context on valuable judgment, and avoiding unnecessary waits for work that can finish in parallel. As the number of models increases, results become worse without this principle.

1. Why You Should Abandon the One-Session-Does-Everything Approach

Context is not a shared workbench

[Why] When one session explores an entire repository and then implements and tests a change, the many details read earlier displace later judgment. Minor error messages and obsolete hypotheses pile up on the same workbench. Eventually, the model responds more readily to the latest log than to the core requirement.

[Rule] Keep only requirement interpretation, priorities, acceptance criteria, and final decisions in the commander's context. Send bulk file exploration, repeated edits, and long test logs to worker contexts.

[Command/Code] As shown below, the commander passes a task specification and tells the worker to read only the necessary files.

```
perl -e 'alarm shift; exec @ARGV' 1800 codex exec -m gpt-5.6-sol -c
    model_reasoning_effort="high" "TASK: Fix the billing status display error. CONTEXT:
    Read src/billing/status.ts and tests/billing/status.test.ts first. SCOPE: Fix the state
```

```
    transitions and the tests. OUT-OF-SCOPE: Do not do any other refactoring. VERIFY: npm
    test -- status. REPORT: Print DONE: <summary> on the last line. Do not ask questions;
    implement immediately. Do not wait for approval. If anything is unclear, proceed with
    reasonable defaults and report your assumptions after completion."
```

[Failure signal] The commander reads the same log several times, or the worker reports lengthy repository background instead of what it changed. Alternatively, a task conversation becomes so long that you have to search for the original acceptance criteria.

[Recovery] Stop the current task and rewrite the goal, allowed files, and verification command in no more than five lines. Give only that specification to a new worker. Do not paste the entire verbose explanation from the previous conversation.

Waiting is the most expensive idle time

[Why] If the commander also waits while an agent runs tests, every model—not just one person—is tied to the same bottleneck. Independent documentation, implementation, research, and testing have no reason to wait for one another.

[Rule] Delegate tasks concurrently when their files and acceptance criteria do not overlap. Never let two or more workers modify the same file directly. Serialize conflicting tasks or compare them in separate worktrees.

[Command/Code] The following example sends three non-overlapping assignments. In actual work, narrow the file scope further in each specification.

```
perl -e 'alarm shift; exec @ARGV' 1800 codex exec -m gpt-5.6-sol -c
    model_reasoning_effort="high" "TASK: Implement API error handling. CONTEXT: Read and
    modify src/api/error.ts only. VERIFY: npm test -- api-error. REPORT: DONE or FAILED."
perl -e 'alarm shift; exec @ARGV' 1800 codex exec -m gpt-5.6-terra "TASK: Update the
    configuration documentation. CONTEXT: Modify docs/configuration.md only. VERIFY: npm
    run lint:docs. REPORT: DONE or FAILED."
glm -p --permission-mode acceptEdits "TASK: Strengthen the unit tests. CONTEXT: Modify
    tests/api/error.test.ts only. VERIFY: npm test -- api-error. REPORT: Last line DONE or
    FAILED."
```

[Failure signal] Several workers are running, but `git diff` shows the same file changing repeatedly, or one worker's changes break another worker's tests. Conversely, there are three independent items, but only one worker is active while the others are idle.

[Recovery] Create a file ownership table immediately. Stop one overlapping task, and separate the work into worktrees if the changes need to be compared. Give idle workers independent quality tasks such as documentation review, test coverage, or adversarial review instead of duplicating the implementation.

A completion report is not evidence

[Why] A worker can describe a plan and stop, miss a test failure, or print `DONE` incorrectly. On 2026-08-18, a codex worker exited after leaving only an approval request and no implementation result. The commander mistook this for completion, which cost time.

[Rule] The commander does not approve work based only on the report sentence. Mark work complete only after reading the result body, inspecting the diff, and directly checking the exit code of the specified verification command.

[Command/Code] For final confirmation, read the end of the log and the exit code together instead of reading a long log.

```
(
  set -o pipefail
  npm test -- api-error 2>&1 | tail -n 5
  test_status=$?
  printf 'exit=%s\n' "$test_status"
  exit "$test_status"
)
git diff --stat
git diff -- src/api/error.ts tests/api/error.test.ts
```

[Failure signal] The worker says “The tests passed,” but provides no command, changed files, or exit code. Or the diff includes files outside the specified scope.

[Recovery] The commander runs the verification command once. If it fails, send the result to the worker and ask it to run a fix loop. Preserve out-of-scope changes in a separate patch before removing them from the current task, then strengthen the specification’s exclusions to prevent recurrence.

2. Why External CLI Workers Instead of Claude’s Internal Subagents?

Claude Code has a native feature for starting subagents inside a session through the Agent/Task tool. It is fair to ask, “If it already runs in parallel, why use external CLIs?” The answer is that this book is not aiming for parallel processing alone. It aims to place **limits, persistence, and failure decisions outside the session**.

Native subagents compared with external CLI workers

[Why] Native subagents run under the same Claude subscription account and within the same session as the commander. That is convenient, but it also means they share the commander’s limits and fate. If you use them without a selection rule, the result is the opposite of what you wanted: “Why is my Claude allowance disappearing at the same rate even though I parallelized the work?”

[Rule] The default is an external CLI worker. Use a native subagent only when the disadvantages in the following table are smaller than its benefits for the current task.

Comparison	Native subagent (Agent/Task)	External CLI worker (codex·GLM·agy)
Distributed limits	No—the commander's Claude subscription allowance is consumed	Yes—each worker uses a separate provider or subscription allowance
Model diversity	Selection is limited to the Claude family	Assign by provider strengths across the GPT family (codex), GLM, and the Gemini family
Context isolation	The subagent carries the exploration burden and returns only a summary. However, the returned report still accumulates directly in the commander conversation	A completely separate process and log file. The commander reads only the conclusion and selected log tail
Background persistence	Managed inside the commander session	Can be managed through a separate process and log. Survival after terminal exit is not automatic, so a monitoring wrapper or supervisor is required
Failure isolation	Failure is returned as an in-session error. If the account allowance is exhausted, every subagent stops at the same time	Isolated per process—judge each one by exit code and log, and one worker's exhausted allowance does not affect the others
Cost	Entirely Claude plan tokens	Each worker uses its own plan. Claude tokens are reserved for specifications, conclusions, and verification

[Command/Code] Submit this command with `run_in_background: true`; record the returned harness task ID and log path. The harness reports termination and its exit code. This does not claim persistence after session loss.

```
mkdir -p .logs
perl -e 'alarm shift; exec @ARGV' 1800 codex exec -m gpt-5.6-terra "<brief>" \
  < /dev/null > .logs/1420_terra_filter.log 2>&1
```

[Failure signal] You parallelize implementation with native subagents to save allowance, but the Claude weekly allowance decreases at the same rate. Or you restart the session, and traces of active subtasks disappear with the conversation.

[Recovery] Move implementation, exploration, and review other than short read-only lookups to external CLI workers. If the session is still alive, rerun the task through an external call that leaves a log file.

Three cases where native is better.

To be clear, native subagents are better than the structure in this book in the following three cases.

1. **Short read-only lookups.** For exploration at the level of "Where is this function defined?", the cost of starting a process and opening a log exceeds the benefit. Native results return directly to the conversation, so there is also no need to manage log files or apply the discipline of reading the result body described in Chapter 5.

2. **Subtasks that need the commander conversation's context.** Native subagents are useful for work that must inherit the parent session's context, such as applying details from a plan just created to several places. With an external CLI, you must rewrite that context as a specification, and distortion can occur during the transfer.
3. **When you do not yet have other subscriptions.** If you do not have codex or z.ai accounts, or have not set up wrappers, native subagents are the only parallel option. They are sufficient for small concurrent exploration tasks and avoid additional account costs and wrapper maintenance.

The two approaches are not opposed. The GLM wrapper (`glm -p`) is an example of connecting the claude CLI's headless call to another provider to create an external worker, as explained in Chapter 2. In this book's actual operating model, native subagents handle routine lookups while external CLI workers handle implementation, exploration, and review.

3. What Claude Should Do Directly and What It Should Delegate

Preventing the commander from doing everything does not make Claude a passive assistant. Instead, it reserves Claude for the most expensive judgments. The following table is a starting point. You can change it for each project, but it is safer to leave a one-line reason for every exception.

Type of work	Default owner	Reason	Evidence checked by the commander
Restating requirements and acceptance criteria	Claude	Ambiguity must be resolved once	Goal, scope, exclusions, verification command
Task decomposition and file ownership	Claude	Conflicts and dependencies must be identified first	Specifications sized for 3-15 minutes
Bulk code exploration	agy or GLM	Protects the commander's context	File list and evidence summary
Difficult debugging	codex Sol	Requires deep reasoning and implementation together	Reproduction, fix, and test results
General feature implementation	codex Terra or GLM	Executes a clear specification quickly	Diff and verification exit code
Mechanical repetition	codex Luna	Suited to renaming and boilerplate	Change list and lint result
Documentation and research	agy	Easy to send as independent support work	Sources, changed files, and conclusion
Final adversarial review	codex Sol	Reduces implementer confirmation bias	Bugs, edge cases, regressions, and security checks
Commit and deployment approval	Claude	Keeps full context and responsibility in one place	Verification record and deployment success marker

Planning and specifications belong to the commander

[Why] Implementers move toward the fastest way to solve the given problem. But if you decide what to build, what not to build, and which failures are unacceptable during implementation, those decisions will drift. If each worker interprets them independently, the results will not integrate.

[Rule] Before calling a worker, the commander fixes the goal, context, scope, exclusions, completion criteria, verification command, and report format in one brief. If the implementation does not branch enough to require a question, state a reasonable default and proceed.

[Command/Code] The following format is a work contract, not a tool-specific format.

```
TASK: Let the user mark notifications as read.
CONTEXT: Read src/notifications/service.ts and tests/notifications/service.test.ts.
SCOPE: Implement only read-state persistence and its unit tests.
OUT-OF-SCOPE: No UI changes, database migrations, or unrelated refactoring.
DONE-CRITERIA: Processing the same notification twice is safe, and the existing tests pass.
VERIFY: npm test -- notifications
REPORT: Last line DONE: <change and verification summary>. If it fails, FAILED: <reason>.
```

[Failure signal] The worker asks, “What UI do you want?” or “Please approve the plan before I proceed,” and then stops. This is not courtesy. It is a gap in the execution contract.

[Recovery] Add “Do not ask questions; implement immediately. Do not wait for approval. If anything is unclear, proceed with reasonable defaults and report your assumptions after completion” to the brief. However, do not use this sentence to automate irreversible actions such as payments, account creation, or data deletion.

Implementation and repetition belong to the workers

[Why] Opening files, reproducing errors, and repeating tests consume more execution time than concentration. If the commander does these tasks directly, it has no time to design the next task. Workers must complete their own edit-test loops to preserve the parallel structure.

[Rule] Give each worker both the files it may modify and a command it can use for self-verification. The commander does not intervene and fix every failure directly. It asks the same worker to retry at most twice, including the failure log and scope.

[Command/Code] For reproducibility, make the codex model and permissions visible in every call.

```
perl -e 'alarm shift; exec @ARGV' 1800 codex exec -m gpt-5.6-terra "TASK: Fix empty-value
    handling in the search filter. CONTEXT: src/search/filter.ts and
    tests/search/filter.test.ts. SCOPE: Handle empty strings and whitespace only.
    OUT-OF-SCOPE: Do not change the API format. VERIFY: npm test -- filter. REPORT: Last
    line DONE or FAILED."
```

[Failure signal] The worker stops after one test fails, repeats the same error twice, or makes broad changes in other directories during the task.

[Recovery] After the first failure, add the necessary portion of the complete error and the expected behavior to the brief, then retry. After the second failure, switch to a worker with different strengths and pass along a one-line explanation of the earlier failure. Do not make a third call with identical input.

Final decisions and commits belong to the commander

[Why] Results from several workers can be partially correct yet collectively contradictory. One worker may add tests while another changes the API contract. A commit records that combination in the product's history, so it requires one decision-maker.

[Rule] The commander decides whether to commit or deploy only after checking the change scope, test exit codes, and review results. Workers may suggest commit messages, but they do not have approval authority.

[Command/Code] Add an independent review for non-trivial changes.

```
perl -e 'alarm shift; exec @ARGV' 1800 codex exec -m gpt-5.6-sol -c
    model_reasoning_effort="high" "Adversarially review the current changes. Check for
    bugs, edge cases, regressions, and security. Fix the problems you find directly and run
    npm test -- notifications. Print DONE: <review and verification summary> or FAILED:
    <reason> on the last line."
```

[Failure signal] The review repeats the implementer's summary or merely says the tests passed without checking side effects.

[Recovery] State the files to inspect, the intent of the change, and prohibited behavior in the review prompt. If the reviewer changes the same files too broadly, remove its edit permission or have it report findings only, then assign those findings to a separate implementation worker.

4. The Worker Pool Is Not a Priority Chain

It may seem convenient to line workers up as "If Sol fails, use Terra; if Terra fails, use GLM," but that is poor operation. It deliberately leaves other workers idle while one worker runs. The pool in this book is an aptitude map, not a reserve list.

Build an aptitude allocation table first

[Why] Even within feature implementation, some tasks require many design decisions, while others mainly involve renaming and formatting. If you allocate work based on model names or a vague sense of remaining allowance, you will waste an excessive model on simple work or throw a difficult error into a weak context.

[Rule] Assign work based on difficulty, file-conflict risk, repetition, and independence. Do not guess limits in advance or decide, "This week, we will use only this model." Move a task only after an actual limit error occurs.

[Command/Code] The calls in the table below match interfaces verified in this environment. Readers should replace the project paths and briefs.

Worker	Aptitude	Example call	Avoid assigning
codex Sol	Difficult debugging, architecture-sensitive implementation, adversarial review	`codex exec -m gpt-5.6-sol -c model_reasoning_effort="high" "<brief>"`	Simple bulk filename changes
codex Terra	Clear general implementation, test changes	`codex exec -m gpt-5.6-terra "<brief>"`	Simultaneous edits to the same file as another worker
codex Luna	Boilerplate, lint, mechanical repetition	`codex exec -m gpt-5.6-luna "<brief>"`	An entire feature with many design choices
GLM	Broad implementation, bulk work with long context	`glm -p --permission-mode acceptEdits "<brief>"`	Calling a metered endpoint directly
agy	Research, code exploration, documentation, supporting review	`agy --dangerously-skip-permissions -p "<brief>"`	Broad repository traversal without a required conclusion

[Failure signal] You repeatedly retry a difficult bug with Luna, or wait for Sol to handle one documentation file. Or you discard a valid GLM result merely because its output contains a model-name warning.

[Recovery] Classify the cause as an aptitude mismatch, an incomplete specification, or an environment error. If it is an aptitude mismatch, move only that task to another worker. GLM's `[claude-code:unrecognized_model]` warning is informational when the response and exit code are normal, so do not treat that string alone as failure.

No idle workers does not mean using as many as possible

[Why] "Eliminate idle workers" is easily misunderstood as assigning five workers to the same code. That kind of parallelism increases only conflicts, duplicated cost, and comparison work.

[Rule] The no-idle principle applies when independent, valuable work exists. If there is only one implementation task, assign the remaining workers work with separate file ownership, such as test design, documentation accuracy review, impact exploration, or adversarial review.

[Command/Code] Send an exploration task that can run alongside implementation with an explicit read scope.

```
agy --dangerously-skip-permissions -p "TASK: Investigate the blast radius of the
    notification read-state change. CONTEXT: Read src/notifications and docs/api.md only.
    SCOPE: Report the call sites, API contract, and regression risk in a table.
    OUT-OF-SCOPE: Do not modify files. DONE-CRITERIA: The affected files and supporting
    evidence are present. REPORT: Last line DONE: <summary>."
```

[Failure signal] Two workers produce nearly identical patches, but no adoption criteria exist. Or starting new workers takes longer than comparing their results.

[Recovery] Use duplicate implementations only for important, difficult changes. When comparing two implementations, use separate worktrees and select one using the same criteria: correctness, tests, change scope, and maintainability.

Limits are observations, not planning inputs

[Why] "This model should have allowance left this week" is a guess made without knowing actual usage, account status, or provider policy. If you allocate work based on that guess, capable workers remain idle and pointless detours continue until an error finally occurs.

[Rule] Allocate by aptitude before the call. Reassign only after an actual limit, rate-limit, or authentication error is confirmed. Do not hard-code per-model limit numbers in team rules.

[Command/Code] The following is a concise reassignment specification that preserves the failure context.

```
TASK: The same search filter fix as before.
CONTEXT: The previous worker could not run due to a rate limit. Read src/search/filter.ts
    and the test file only.
SCOPE: Implement exactly the existing acceptance criteria.
OUT-OF-SCOPE: Do not overwrite the previous worker's unfinished changes based on guesses.
VERIFY: npm test -- filter
REPORT: Last line DONE or FAILED.
```

[Failure signal] You frequently change assignments despite the absence of an error, or duplicate one task across several workers after seeing a single warning line.

[Recovery] Check the process response, exit code, and result body. Classify a worker as failed only when it does not respond or exits with a nonzero status. Do not infer failure from a warning string, log length, or model name alone.

5. The Commander's Five-Line Rule

Complex operating rules are hard to remember for every task. The commander therefore checks the following five lines every time. This is not a model-selection checklist. It is the minimum contract for deciding whether a task can be executed.

1. Is the goal one sentence?
2. Is the scope of files to modify or read defined?
3. Does it state what must not be done?
4. Is there a verification command that proves completion?
5. Is the final report defined as `DONE` or `FAILED`?

Do not execute without all five lines

[Why] With a weak specification, a worker asks questions, expands the scope on its own, or describes a plan and stops. Calling this a model-performance problem guarantees recurrence.

[Rule] If any of the five lines is missing, complete the brief first. For an obvious correction that fits within five lines, such as a single typo, the commander may handle it directly.

[Command/Code] Compressed into an actual prompt, the five lines look like this.

```
TASK: Make an empty search query produce no server error.
CONTEXT: src/search/filter.ts, tests/search/filter.test.ts.
SCOPE/OUT-OF-SCOPE: Handle empty strings and whitespace only; do not change the API format.
VERIFY: npm test -- filter.
REPORT: Last line DONE: <summary> or FAILED: <reason>. Implement immediately and do not
    wait for approval.
```

[Failure signal] The worker says “complete” but does not say what it verified, or it reads the entire repository while trying to locate files.

[Recovery] Instead of making the requirement longer, specify filenames and the verification command more precisely. If you do not know the scope, first assign an exploration-only worker to return just the file list.

6. Completion and Reassignment Are Separate Operating Rules

At the role-design stage, remember only two boundaries. Launch every worker with `run_in_background: true`; on notification, read its exit code and result body. codex logs may include `tokens used` as an optional supporting diagnostic, never a completion gate. After one changed retry that addresses the cause, move a failed task to a worker with different strengths. See Chapter 5 for detailed state transitions and incident analysis, and Chapter 7 for the reassignment procedure.

7. The Safe Scope of Command

As the 2026-08-20 Chrome profile incident showed, accounts, payments, KYC, and default browser profiles are outside worker authority. Use only isolated browser profiles, close them after the task, and check only whether secret values exist. See Chapter 9 for the detailed denylist and recovery procedure.

Model names are part of the contract

[Why] When a model name or default changes, the same specification produces a different kind of execution. In particular, if a lower-tier model is silently substituted, the result is difficult to reproduce.

[Rule] Specify the required model ID and permissions in each call. Record names such as `gpt-5.6-sol/terra/luna`, `glm-5.3`, and `claude-fable-5/opus-5/sonnet-5` as part of the contract at

execution time instead of relying only on wrapper defaults.

[Command/Code] Call GLM through the provided wrapper. The wrapper sets `ANTHROPIC_BASE_URL`, passes the authentication token, and configures `CLAUDE_CODE_MAX_CONTEXT_TOKENS` for the process. Never write the key itself in the text or repository.

```
GLM_MODEL=glm-5.3 glm -p --permission-mode acceptEdits "<brief>"
```

[Failure signal] Someone guesses and changes the model name, or adds model mapping to a configuration file to "fix" a normal GLM warning.

[Recovery] Check the actual wrapper output and exit code first. Do not stop a successful call because of an `unrecognized_model` warning alone. Do not conceal the issue with enterprise mappings such as `modelOverrides` in `settings.json`.

8. The Operating Board to Create on Day One

The commander structure does not require a large dashboard. One text table is enough. What matters is that every task separately tracks "running" and "result read." The first is process state; the second is the commander's responsibility state.

ID	Goal	Owner	File ownership	Verification	Status	Result read
A	Fix API error	Sol	`src/api/error.ts`	`npm test -- api-error`	In progress	No
B	Expand tests	GLM	`tests/api/error.test.ts`	Same test	Pending	No
C	Investigate contract	agy	Read-only	Report	In progress	No
D	Update documentation	Luna	`docs/api.md`	Documentation lint	Pending	No

[Why] Without this table, the commander remembers only the most recently viewed log. It misses tasks that finished earlier, waits for workers that have already exited, or assigns new work against the same file.

[Rule] Add a row when starting a worker and update the result-read column after it exits. Before reporting progress to the user, read the result body for every row that was started.

[Command/Code] The table can live in a Markdown file, issue, or note. Keep the status vocabulary small, such as `Pending`, `In progress`, `Verification`, `Complete`, and `Failed`.

[Failure signal] You have to search logs to answer, "Who is doing what?"

[Recovery] Do not build a new tool. Reconstruct only the current work in a table. One line each for running processes, exited processes, and unread results is enough.

Checklist

- Have you separated Claude's responsibility for the goal, scope, exclusions, verification, and approval decision?
- Does every worker brief define the file scope and a `DONE` or `FAILED` report format?
- Are only independent tasks assigned in parallel, with simultaneous edits to the same file prevented?
- Is allocation based on task aptitude rather than guesses about limits?
- If you chose native subagents, did you verify that the reason is not a mistaken expectation of distributed limits?
- Do you use exit code plus reading the body for every worker, with `tokens used` only an optional codex diagnostic?
- Have you read the result body, diff, and verification-command exit code for every worker?
- Have you excluded accounts, payments, the default browser profile, and secret values from worker scope?

Three common failures

1. **The commander takes on implementation too.** It loses the capacity to design the next task. Send file edits and test loops to a worker with a brief.

>

2. **Every worker is assigned the same patch.** Duplication without comparison criteria is not parallelism. Divide file ownership, or use duplicate implementations only for important changes in separate worktrees.

>

3. **An exit message is trusted as completion.** A report is not evidence. Completion requires the result body, diff, and verification exit code.

2. 30-Minute Setup: Install, Authenticate, and Wrap Five Workers

> **Revision — 2026-09-09:** Completion now uses harness exit code plus reading the result body, with notification-based waiting; marker gates and polling were retired to prevent false STALL decisions.

In this chapter, you will build the minimum environment needed to call one commander and five types of execution workers. The goal is not an impressive dashboard. The goal is to confirm that each worker receives a short smoke test and returns a completed result. Even after installation, always check the current CLI help again before starting work. Tool defaults and model lists can change.

The examples in this book never print secret values. Manage actual values through shell startup files or the provider's secure credential mechanism. Never put them in a repository, document, or task prompt.

> **Worker-call convention:** The CLI examples below show command payloads. For each headless worker or smoke test, pass that payload to Chapter 5's `worker_launch.sh` with a unique attempt name and submit the launcher as the only command of a `run_in_background: true` tool call. Read the notification exit code and matching log separately. Installation and interactive authentication are setup steps, not worker runs.

1. Define the Boundaries Before Setup

Fix the working directory first

[Why] A headless worker interprets the current directory as its project. If you run it from an unrelated parent directory, it may fail to find files or read and modify an unintended repository. Non-interactive codex calls in particular can be affected by whether the location is a Git repository.

[Rule] Move to the project root before running each worker. Keep result files and logs inside that project. Do not include working-directory or Git-check bypass options in examples unless they are confirmed in the `codex exec` help provided with this book.

[Command/Code] The following is the smallest preparation for checking the current working location.

```
pwd
git rev-parse --show-toplevel
```

Specify the project root with `cd`. This approach does not rely on a separate working-directory option.

```
PROJECT_ROOT=$(git rev-parse --show-toplevel) || exit 1
cd "$PROJECT_ROOT" || exit 1
codex exec -m gpt-5.6-luna "TASK: Check the Markdown headings in the current folder.
    CONTEXT: Read Markdown files only. SCOPE: Do not modify anything; produce only a list
    of problems. VERIFY: None. REPORT: Last line DONE: <summary>."
```

[Failure signal] The worker mentions files unrelated to the project, or a Git-repository validation error prevents the actual task from starting.

[Recovery] Check `pwd` in a new terminal and move to the project root. If the task requires a Git repository, rerun it from a location where `git rev-parse --show-toplevel` succeeds. Do not guess and append options that are absent from both the local `codex exec --help` and the provided help text.

Check only whether secret values exist

[Why] Once an API key or token appears in a log, shell history, task prompt, or screenshot, it is difficult to recover control of it. Workers may summarize input or read files while solving a problem, so passing a file containing a key as context is also dangerous.

[Rule] Diagnose authentication by checking only `SET` or `missing`, never the value. Do not put keys in a repository `.env`, documentation, or code block. Even when loading them through a shell startup file, use a private file separate from public repositories.

[Command/Code] This command does not print the value itself.

```
if [ -n "$ZAI_API_KEY" ]; then
  echo "ZAI_API_KEY=SET"
else
  echo "ZAI_API_KEY=missing"
fi
```

[Failure signal] You are about to run a command such as `echo $ZAI_API_KEY`, `env`, or `printenv` that prints the full value, or paste the key as a fixed string into a configuration file.

[Recovery] Close the output immediately and do not share it. If exposure is possible, revoke and replace the key with the provider. From then on, run only presence checks and never include `.env` in a worker's CONTEXT.

2. Installation and Subscription Boundaries

Install four tools and verify their versions

[Why] The five worker types in this book run through four executables: the codex CLI (Sol, Terra, and Luna use the same executable with different models), Claude Code (the commander and the base of the GLM wrapper), and either Gemini CLI or Antigravity CLI (`agy`). Installation commands can change, so the commands below include the versions actually installed in the author's environment when this book was written, in August 2026.

[Rule] After installation, always use `--version` to confirm that the executable responds. Then use the smoke test in the next section to verify that it accepts a prompt and returns a result. If package names or login procedures differ from the examples below, the provider's current official documentation takes precedence.

[Command/Code]

```
# 1) codex CLI — OpenAI. Use it by logging in with a ChatGPT subscription (Plus/Pro/Team,
    etc.)
npm install -g @openai/codex
codex login            # A browser opens for login with a ChatGPT account
codex --version        # Author's environment: codex-cli 0.147.0

# 2) Claude Code — Anthropic. Claude Pro/Max subscription or API key
#    Use either the official installer (recommended) or npm
curl -fsSL https://claude.ai/install.sh | bash
# Or: npm install -g @anthropic-ai/claude-code
claude --version       # Author's environment: 2.1.251 (Claude Code)
claude                 # Login instructions appear on first run

# 3) Gemini CLI — Google. Free allowance is available by signing in with a personal Google
    account
npm install -g @google/gemini-cli
gemini --version       # Author's environment: 0.51.0

# 4) agy — Google Antigravity CLI (Gemini family). Used in this book for research and
    exploration
#    Follow Antigravity's official installation and login instructions. If unavailable, it
    can be replaced with gemini from step 3 (see Section 5)
agy --version          # Author's environment: 1.1.22
```

Tool	Executable	Account/subscription	Role in this book
codex CLI	`codex`	ChatGPT subscription login	Sol/Terra/Luna workers (select model with `-m`)
Claude Code	`claude`	Claude subscription	Commander + base of the GLM wrapper
GLM wrapper	`glm` (buyer file `templates/glm_wrapper.sh`)	z.ai Coding Plan subscription + `ZAI_API_KEY`	Bulk parallel implementation worker
Gemini CLI / agy	`gemini` / `agy`	Google account	Research, exploration, and documentation worker

The GLM wrapper is not a separate installation. It is **a shell script that runs Claude Code through the z.ai endpoint**. Copy the buyer file `templates/glm_wrapper.sh` to a location on your PATH such as `~/.local/bin/glm`, then make it executable. The complete script appears in Section 4 and Chapter 12.

```
cp templates/glm_wrapper.sh ~/.local/bin/glm && chmod +x ~/.local/bin/glm
```

[Failure signal] `--version` responds, but the smoke test ends at a login screen or with a permissions error. Or `codex --version` works, but `codex exec` exits immediately with "Not inside a trusted directory" (see `--skip-git-repo-check -C <path>` in Section 3).

[Recovery] Separate the diagnosis into four stages: executable presence (`command -v`), version response, login, and smoke test. Record where it stops. For authentication issues, return to each tool's `login` or first-run instructions. Check the current `--help` output before changing commands from the book.

Check executable presence first

[Command/Code] Check only for executable presence without printing values or account names.

```
for worker_command in codex claude glm agy; do
  if command -v "$worker_command" >/dev/null 2>&1; then
    printf '%s=FOUND\n' "$worker_command"
  else
    printf '%s=missing\n' "$worker_command"
  fi
done
```

`FOUND` means only "installed." It does not mean authentication, model access, or the subscription is ready. Judge those conditions only from the smoke test's exit code and conclusion body.

3. codex CLI: Calling Sol, Terra, and Luna Separately

codex is the core implementation and review worker in this structure. It uses the same `codex exec` command, but you specify the model and reasoning effort according to task difficulty. If you rely on default settings, a different shell or machine may apply an effort level other than the one you expected.

`gpt-5.6-sol`, `gpt-5.6-terra`, and `gpt-5.6-luna` are not separate executables. They are model IDs passed to `codex exec -m`. The date of the model names used in this book is stated once in the note in Chapter 0.

The basic form of a non-interactive call

[Why] An interactive window is useful when a person is responding, but it can wait for input when you dispatch several tasks at once and collect their logs. `codex exec` accepts one prompt and runs

non-interactively, which suits a worker pool.

[Rule] Specify the model with `-m` for every task. For Sol, explicitly add `-c model_reasoning_effort="high"` to remove differences in default effort. For sandboxing or write permissions, do not copy flags that were not confirmed in the shortened help supplied with this book. Check the currently installed help and project policy separately.

[Command/Code] Use Sol for difficult error analysis and fixes.

```
codex exec -m gpt-5.6-sol -c model_reasoning_effort="high" "TASK: Fix the date boundary
    error. CONTEXT: Read src/date/range.ts and tests/date/range.test.ts only. SCOPE: Fix
    the time-zone boundaries and the unit tests. OUT-OF-SCOPE: Do not update dependencies.
    VERIFY: npm test -- date-range. REPORT: Last line DONE: <change and verification
    summary> or FAILED: <reason>. Do not ask questions; implement immediately. Do not wait
    for approval. If anything is unclear, proceed with reasonable defaults and report your
    assumptions after completion."
```

[Failure signal] You omit the model because Sol is slow, or a low default effort produces only a shallow change for a complex error.

[Recovery] Inspect the call command before the execution log. Confirm that it included `-m gpt-5.6-sol` and `-c model_reasoning_effort="high"`. If the specification is sufficient but the task still fails twice, do not repeatedly call Terra. Transfer it to a worker with different strengths and include the failure evidence.

Terra and Luna are different workbenches, not assistants

[Why] If you use only Sol, other independent tasks accumulate in a queue. Assign clear general implementation to Terra and formal, repetitive changes to Luna so the commander does not become the bottleneck.

[Rule] Give Terra bounded work such as one feature and one test. Give Luna work with few design decisions, such as documentation link cleanup, boilerplate, or lint fixes. Neither should modify the same file at the same time as Sol.

[Command/Code] The following are examples of general implementation and mechanical checking.

```
codex exec -m gpt-5.6-terra "TASK: Add an input validation message. CONTEXT:
    src/forms/validate.ts and tests/forms/validate.test.ts. SCOPE: Add only the empty-email
    error. OUT-OF-SCOPE: Do not overhaul the UI wording. VERIFY: npm test -- validate.
    REPORT: Last line DONE or FAILED."

codex exec -m gpt-5.6-luna "TASK: Replace the outdated command names in the documentation.
    CONTEXT: Modify docs/cli.md only. SCOPE: Change old-command to new-command.
    OUT-OF-SCOPE: Do not change the example structure. VERIFY: npm run lint:docs. REPORT:
    Last line DONE or FAILED."
```

[Failure signal] Luna stops to ask for a design decision, or Terra restructures the architecture across the repository.

[Recovery] Break Luna's task into something more mechanical. Reduce Terra's scope to individual files. Return any part that requires design decisions to Sol or the commander's specification stage.

Check the default effort in config.toml

[Why] The default effort in `~/.codex/config.toml` may be set for convenience, but it can differ from the quality level expected by the worker pool. Results for complex work in particular can vary by environment when the caller does not specify effort.

[Rule] Treat config.toml only as a personal default. For important Sol calls, always override it with `-c model_reasoning_effort="high"` or the required value. Before changing the global default, check what existing automation expects.

[Command/Code] Because the file may contain secrets, do not share it in full. Search only for the relevant keys.

```
rg -n "model_reasoning_effort|model" ~/.codex/config.toml
```

[Failure signal] The only evidence is "It worked on my machine," and the model and effort used cannot be reproduced.

[Recovery] Put the model and effort in every important command. A config.toml change can affect several projects, so do not turn one project's special requirement into a global default.

codex smoke test

[Why] Installation must be verified by whether the selected model accepts a non-interactive prompt and returns a complete result, not merely whether the command name exists.

[Rule] Use a short read-only task that does not modify files. Do not provide product code or secret files as context.

[Command/Code] Inside a Git repository, run the following.

```
codex exec -m gpt-5.6-luna "TASK: Report only the three top-level file names in the current
    project. CONTEXT: Read the top-level directory only. SCOPE: Do not modify files.
    VERIFY: None. REPORT: Last line DONE: <file list summary>."
```

[Failure signal] The process exits, but its body contains only an approval request without the requested result. Missing `tokens used` is not a failure condition; it is an optional codex diagnostic.

[Recovery] Add immediate execution and the `DONE` format to the end of the prompt, then retry once. If it fails again, record the error and current directory and reassign the work to another worker. Do not simply assume success.

4. Headless Claude Code Workers and the GLM Wrapper

Run Claude Code once with -p

[Why] When Claude Code is used as a worker, opening an interactive window can leave it waiting for the next input. For parallel work, passing the prompt as an argument and receiving the entire run through exit is more predictable.

[Rule] Use `-p` for headless calls. Specify `--permission-mode acceptEdits` for modification tasks. Use settings that also broaden automatic command-execution permission only in trusted projects, and narrow the file scope first.

[Command/Code] Keep a read-only smoke test short.

```
claude -p "TASK: Report only the number of Markdown files in the current folder. CONTEXT:
    Read the current folder only. SCOPE: Do not modify anything. VERIFY: None. REPORT: Last
    line DONE: <count>."
```

For a GLM task that requires modifications, specify permissions through the wrapper.

```
glm -p --permission-mode acceptEdits "TASK: Fix the broken links in the README. CONTEXT:
    Read and modify README.md only. SCOPE: Fix only link notation that can be verified.
    OUT-OF-SCOPE: Do not rewrite the content. VERIFY: rg -n 'old-link' README.md. REPORT:
    Last line DONE or FAILED."
```

[Failure signal] The worker exits after describing only a plan, stops at a permission request, or requires a person to keep watching an interactive window before it can continue.

[Recovery] Recheck the position of `-p` and `--permission-mode acceptEdits`. Add allowed files and "do not wait for approval" to the prompt. If the risky task still requires a permission boundary, do not automate it; hand it to a person.

The GLM wrapper fixes the endpoint and context size

[Why] When calling GLM through the Claude Code interface, manually setting the endpoint, passing authentication, selecting the model ID and small-model default, and choosing the context limit every time creates errors. In particular, you must prevent accidental use of a metered endpoint or a silent switch to a smaller helper model.

[Rule] This chapter follows the behavior in `build/glm_wrapper.txt`. Call GLM only through the `glm` wrapper. The wrapper sets the subscription route `https://api.z.ai/api/anthropic` and related environment variables for the process, and pins both the default model and small helper model to `glm-5.3`. It never places the credential in code and reads it only from the parent shell's `ZAI_API_KEY`.

[Command/Code] Even when specifying the model, keep the key only in the shell environment.

```
GLM_MODEL=glm-5.3 glm -p --permission-mode acceptEdits "TASK: Clean up the test error
```

```
    messages. CONTEXT: Modify tests/errors.test.ts only. SCOPE: Align only the expected
    messages and assertions. OUT-OF-SCOPE: Do not modify application code. VERIFY: npm test
    -- errors. REPORT: Last line DONE or FAILED."
```

The complete wrapper appears in the buyer file `templates/glm_wrapper.sh` and in Chapter 12. Keep `ZAI_API_KEY` only as an environment variable loaded by a shell startup file. Do not write its value in the script.

[Failure signal] You classify the run as failed solely because the log contains `[claude-code:unrecognized_model]`, or try to fix it by adding the GLM name to `modelOverrides` in `settings.json`.

[Recovery] This can be an informational message caused by Claude Code recognizing only Anthropic model IDs. Check the response body and exit=0 first. Actual failure means no response or exit≠0. Do not detour through the metered `/api/paas/v4` endpoint or add arbitrary model mappings.

GLM smoke test

[Why] The wrapper handles environment variables and an external endpoint together, so checking command presence alone cannot establish that the authentication, model, and response path are correct.

[Rule] Ask for a one-sentence answer in the smoke test, and do not include secret information in the output. Even when warnings appear, judge the run by its exit code and conclusion body.

[Command/Code] The following check does not modify files.

```
glm -p "TASK: Print the sentence 'worker ready'. CONTEXT: Do not read files. SCOPE: Do not
    modify files or make external calls. VERIFY: None. REPORT: Last line DONE: smoke test."
```

[Failure signal] There is no response, or the process exits with a nonzero status. A message saying the key is missing means the environment lacks the key; it is not the same kind of issue as a model warning.

[Recovery] Check only whether `ZAI_API_KEY=SET` and inspect the private shell startup configuration. Do not copy and paste the value into chat or a log. Record separately whether the failure occurred at the endpoint, key, model, or context-limit stage.

5. agy and Gemini-Family Supporting Workers

agy is the **Google Gemini-family CLI (Antigravity CLI, command `agy`)**. In this book, `agy` means the supporting worker responsible for research, code exploration, and documentation. Its name differs from the standard Gemini CLI command, `gemini`, so do not mistake them for the same executable.

Assign research and documentation to agy

[Why] Codebase exploration and material organization unnecessarily fill an implementation worker's context. Using agy as a separate reading and documentation worker lets Sol and Terra focus on edits and tests.

[Rule] For agy, specify the directories to read, output format, and prohibition on modifications. Use only `--dangerously-skip-permissions` and headless `-p`, which are confirmed in the provided Claude-family help. Use calls that broaden permission boundaries only in trusted projects, and exclude accounts, browsers, and secret files from scope.

[Command/Code] The following assigns an impact analysis.

```
agy --dangerously-skip-permissions -p "TASK: Investigate the call sites of the notification
    API change. CONTEXT: Read src/notifications and docs/api.md only. SCOPE: Report the
    per-file impact and regression risk as a Markdown table. OUT-OF-SCOPE: Do not modify
    files, read secret files, or use the browser. VERIFY: None. REPORT: Last line DONE:
    <summary>."
```

[Failure signal] agy starts making changes or traverses an entire large repository because the research scope was not defined.

[Recovery] State `OUT-OF-SCOPE: Do not modify files` clearly and narrow the read paths to two or three. If the result remains too broad, do not add a new tool to the next call. Reduce the assignment to one question.

Verify Gemini CLI/agy from actual output

[Why] Installation instructions can become outdated, and login state can differ between terminals. The ability to invoke a name does not prove that a prompt was processed.

[Rule] Verify a Gemini-family supporting worker by running a short read-only prompt through the provided `agy` interface. Do not guess a model and specify it with another flag. Use only calls supported by the current help and the organization's wrapper.

[Command/Code] This smoke test requests a safe conclusion body.

```
agy --dangerously-skip-permissions -p "TASK: Reply with the single line 'support worker
    ready'. CONTEXT: Do not read files. SCOPE: Do not modify files, use the browser, or
    touch accounts. VERIFY: None. REPORT: Last line DONE: smoke test."
```

[Failure signal] The process exits without a conclusion body, or an authentication or permission error appears.

[Recovery] Check the exit code and final body. A person resolves authentication through the provider's normal login procedure. The commander does not create or bypass credentials on the person's behalf.

6. Connect It Safely to the Shell Startup File

Keep environment variables in private scope

[Why] To use workers across several terminals, credentials must also be available in new shells. But if you place them in the project `.env` or README, they can spread through Git, backups, collaborators, and worker contexts.

[Rule] A personal shell startup file should contain only initialization that reads from an approved secret store, not the secret value itself. Do not put raw keys directly in `~/.zshrc`, a project `.env`, or documentation. After injection into a new shell, check only whether the value exists.

[Command/Code] For a smoke test without a separate secret store, let zsh read the value without displaying it and keep it only in the current shell. It disappears when the shell exits.

```
read -r -s "ZAI_API_KEY?ZAI API key: "
printf '\n'
export ZAI_API_KEY
```

After applying it, check only for presence.

```
if [ -n "$ZAI_API_KEY" ]; then echo "ZAI_API_KEY=SET"; else echo "ZAI_API_KEY=missing"; fi
```

[Failure signal] You modify the configuration file but see `missing` in a new terminal, or print the actual key to verify it.

[Recovery] Confirm that the active shell is zsh and check again in a new terminal. Do not print the value itself. If a project environment file conflicts with the personal startup file, inspect only which file is loaded, the presence state, and file permissions.

Smoke-test one worker at a time

[Why] If you run five workers concurrently from the beginning, you cannot isolate which authentication, wrapper, or permission boundary failed. During installation, diagnosability matters more than parallel speed.

[Rule] Run smoke tests one at a time, for example in the order codex Luna, Claude Code, GLM, and agy. Read each exit code and conclusion body before moving to the next worker. After installation is verified, parallelize independent tasks in actual work.

[Command/Code] Record the check results in the following table.

Worker	Model or interface	Smoke test	Exit code	Conclusion body	Decision
codex	gpt-5.6-luna	Read-only prompt	Record	Read	Ready/blocked
Claude Code	`claude -p`	One-line response	Record	Read	Ready/blocked

Worker	Model or interface	Smoke test	Exit code	Conclusion body	Decision
GLM	glm-5.3	One-line response	Record	Read	Ready/blocked
agy	`agy --dangerously-skip-permissions -p "..."`	One-line response	Record	Read	Ready/blocked

[Failure signal] The table says only “ran” and omits the exit code and result body.

[Recovery] Withdraw the ready decision and read the end of that worker’s log. Narrow the cause to one category—key, model, cwd, permission, or network—before rerunning it.

7. Operating Rules for the First Parallel Run

Divide file ownership first

[Why] If you send five prompts into the same repository as soon as the workers are ready, the resulting conflict is an operational design problem, not an installation problem. The last worker to save can overwrite another worker’s changes.

[Rule] Begin the first parallel run with three tasks that modify different files. Assign read-only research or adversarial review to additional workers. Duplicate implementations of the same important task belong only in separate worktrees.

[Command/Code] Write down file ownership first.

```
Sol: src/api/error.ts
Terra: tests/api/error.test.ts
Luna: docs/api.md
GLM: read-only impact analysis
agy: read-only regression risk investigation
```

[Failure signal] One worker changes a test file while another worker changes the same assertion.

[Recovery] Stop one task and reassign ownership. If changes have already conflicted, compare both results and verify one as the baseline. Do not trust an automatic merge.

During setup, record only exit codes and conclusions

[Why] Repeating the full completion decision in the setup chapter can easily conflict with the monitoring rules in Chapter 5.

[Rule] In the smoke-test table, record only the process exit code and conclusion body. Use `run_in_background: true` with the bounded launcher in Chapter 5, then read the harness exit code and result body for every worker. codex’s `tokens used` is only an optional supporting diagnostic.

[Command/Code] If you need to reduce output through a pipeline, run it without hiding failure from the original command.

```
(
  set -o pipefail
  npm test 2>&1 | tail -n 5
  verify_status=$?
  printf 'exit=%s\n' "$verify_status"
  exit "$verify_status"
)
```

[Failure signal] You record `tail` succeeding as smoke-test success, or do not read the conclusion from an exited worker.

[Recovery] Withdraw the ready decision for that row and collect the original command's exit code and conclusion again. Use Chapter 5's harness notification flow; do not poll or gate completion on markers.

8. The Standard for Completed Setup

Setup is not complete merely because the tools are installed. It is complete when each worker can process a prompt in a safe directory without exposing secret values, and the commander can judge the result. Model speed and response length are not smoke-test acceptance criteria.

Start actual work with small tasks as well. Send reversible tasks in parallel, such as one documentation file, one unit-test file, and one read-only impact analysis, then have the commander check the diff and exit codes. Only after that run passes should you divide a larger feature across the worker pool.

Checklist

- Did you call every worker from a project root confirmed with `cd`?
- Did you actually install codex, Claude Code, and Gemini CLI and verify the selected subscription and authentication path?
- Did you specify a model ID in codex calls and an effort level for important Sol tasks?
- Did you use `-p` for headless Claude Code tasks and `--permission-mode acceptEdits` for modification tasks?
- Did you call GLM only through the subscription wrapper with `glm-5.3`?
- Did you avoid printing API keys and tokens or placing them in prompts, repositories, and documentation?
- Did you read both the exit code and conclusion body for every smoke test?
- Did you divide file ownership and verification commands before the actual parallel run?

Three common failures

1. **Trusting defaults and omitting the model and effort.** Specify the model ID and required effort for important calls to preserve reproducibility.

>

2. **Misclassifying a GLM warning as failure.** `unrecognized_model` can be informational. Check it together with the response body and exit=0.

>

3. **Printing a key in troubleshooting logs.** Never inspect the value. Check only `SET` or `missing`, and replace the key immediately if exposure is suspected.

End of preview

The full 13 chapters, the real-run appendix, and the 8 templates are available from the purchase link in the GitHub repository README.